

SECURITY ENGINEERING

Email Spam Classification using Machine Learning
Techniques

OUTLINE

ABSTRACT

PROBLEM STATEMENT

INTRODUCTION

LITERATURE SURVEY

METHODOLOGY

APPLICATIONS

RESULTS

FUTURE DIRECTIONS

CONCLUSIONS

REFERENCES

ABSTRACT

With the explosive growth of digital communication, email has become an essential channel for both business and personal communication. However, spam email presents major challenges, affecting performance and system integrity. Traditional methods, such as keyword filtering and domain blacklisting, have proven inadequate due to the limitation of accurately classifying emails as spam or legitimate (ham). This study presents a sophisticated machine learning approach. Methods are used to suggest better spam detection. By analyzing various structural features of the email, such as Cc/Bcc fields, domain information, and headers, the system uses a pre-trained machine learning model with a feedback mechanism to improve classification accuracy. Thus, this new system not only provides robust spam filtering solutions but also adapts to changing spam channels. Protects and improves email and communication efficiency.

INTRODUCTION

Email is a ubiquitous form of communication in business and personal situations, facilitating the rapid transmission of a variety of complex information. Every day, approximately 205 billion emails are sent worldwide, emphasizing the activity each day and world all subject, of email. This emphasizes the important role of communication. However, this feature also comes with important weaknesses. Spam often uses email systems, which not only clutter inboxes but pose security risks through phishing, ransomware and other malicious activity. The availability of SMTP and POP protocols further complicates the problem, and it hides unsolicited and harmful content. This presentation explores a machine learning-based approach to address this common issue, improving the integrity and efficiency of email communication systems with a sophisticated approach to spam detection and effectiveness of the delivery.

PROBLEM STATEMENT

Despite advances in digital communications technology, spam email continues to plague email systems worldwide, causing significant loss of productivity and potential security breaches. Traditional spam detection techniques, such as keyword filtering and domain blacklisting, due to its static nature proved inappropriate due to the inability to adapt to the evolving spam channels and this leads to increased false positives and negatives, emails qualified by the law misidentified as spam and vice versa. The challenge is to design a dynamic and robust system that can accurately distinguish spam from legitimate (ham) email, thereby increasing the effectiveness of email communications and protecting users from potential threats.

LITERATURE SURVEY

As we started our literature survey, we were highly committed to accessing information from various academic papers and journal articles across the internet. Each group member was assigned specific topics within this wide area to make sure a thorough and comprehensive coverage was taken on this subject. We used renowned academic databases like IEEE Xplore and Google Scholar, alongside industry reports and white papers. Thus, this approach underlined the rigorous set-up of access to new and relevant works in the technology and strategies used in spam detection. The next step in our endeavor was to synthesize the relevant literature through critical analysis. We called these weekly meetings in order to discuss and analyze the findings of each article concerning the approaches, as well as comparing and noting common techniques used in spam detection. This collaborative effort was very important because it allowed us to integrate all the different viewpoints and deepen our collective understanding of the subject. By pinpointing gaps in the research and highlighting unique approaches, we identified prevalent patterns to refine our research questions and guide the subsequent direction of our project towards innovative solutions. Concluding our literature survey, we eventually came to a place whereby the literature review findings needed to be formulated in a well-structured and detailed document. We had structured our literature by the used themes, methodologies, and outcomes for easier referencing and retrieval. This document was not just a critical resource for our project; it ensured that we maintained a clear record of all sources and discoveries. Our synthesized literature review provided a firm footing for the development of the proposed model for email spam classification. It actually equipped us to develop educated perceptions and introduced new propositions that are good for advancing our efforts in research and development in this regard.

1. The article "Sorting the Digital Stream: Big Data-driven Insights into Email Classification for Spam and Ham Detection" seeks to establish discrimination criteria between spam and ham, and it explores feature selection techniques like Chi-Square and Pearson Correlation for enhancing classification accuracy[3].
2. A comparative analysis of four machine learning techniques[2], which includes Deep Learning, SVM, Random Forest, and XGBoost for spam detection, is conducted in the article "Comparative Analysis of Deep Learning, SVM, Random Forest, and XGBoost for Email Spam Detection: A Socio-Network Analysis Approach."
3. An advanced email spam detection technique, which is a hybrid method, utilizes multiple algorithms, such as SVM, Neural Networks, in the article "Advanced Email Spam Detection: A Machine Learning Solution." The proposed UCI spam base dataset training focuses on phishing detection[1].

On the other hand, our Proposed Data Science Approach for Email Spam Classification using Machine Learning Techniques document outlines a system architecture that involves the use of machine learning in a specific set of email parameters (e.g., Cc/Bcc fields, header) to classify spam. It emphasizes a feedback mechanism within a pre-trained model to refine classification based on ambiguous and proper outputs, which is different from the analytical and comparative nature of the articles that explore various machine learning methods and their effectiveness broadly.

The three articles would predominantly focus on methodological comparisons and theoretical advancements in machine learning for spam detection, albeit not necessarily pragmatic or system-wide application. Feature selection evaluation seems to be incredibly thorough in the first article, although it may fail to address real-time adaptability or the dynamics of spam evolution[3]. The second, albeit broadly focusing on machine learning analysis, may fail to address real-time integration of these technologies into existing systems[2]. The third article focuses more on phishing spam, with a lack of consideration for user-specific spam variations and feedback mechanisms for ongoing learning[1].

On the contrary, our Proposed Data Science Approach for Email Spam Classification using Machine Learning Techniques provides a structured, user-centered system that changes to adapt to individual variations in spam perception and a feedback loop to enhance classification performance. This approach, based on practical deployment strategies not detailed in the first three articles, instead puts more weight on real-world application over theoretical analysis.

NOVELTY

We are using library in C++ environment for enhancing text processing capabilities which is not discussed in the paper

We are using different stages of text processing are provided preprocessing.cpp for initial data cleaning, TFIDF.cpp for feature weighting), indicating a pipeline that is ready for real-world applications.

We have used different datasets : Bag of words, the updated data set has 4196 entries but contains only two columns

METHODOLOGY

PRE-PROCESSING

- Removal of special characters
It is the removal of unnecessary characters to make the content understandable, such as character lines, special characters from email. This reduces the complexity of the text data and avoids unnecessary noise in the of the dataset.
- Special Character Removal
This step involves the removal of unnecessary characters to make the content easy to understand, such as character lines, special characters from email. This helps to reduce the complex text data and removes noise in the of the dataset.
- Removal of numbers
Numbers are often excluded because they may be less costly to detect spam. Like special characters, numbers can introduce noise into the data unless they are particularly relevant.
- Lowercase Conversion
Converting all text to lower case will improve consistency throughout the data structure.

Feature Extraction:

Identification of Features:

- To, From, Cc, Bcc Fields: metrics such as the sender (From), recipient (To), carbon copy (Cc), and blind carbon copy (Bcc) option in the email header are considered as important features for classification .
- Message-ID: The unique identifier assigned to each email message (Message-ID) is another feature that can be used to differentiate between spam and ham emails .
- Keywords and Punctuations: Commonly used keywords and punctuations in the email body are also extracted as features for classification purposes

Data Collection and Preprocessing

We have used csv file which is a Term-Frequency-Inverse Document Frequency

Which is a table where each row and column has a unique term

Column A: Contains the subject lines of email

Column B: Includes body text of emails.

Column C: Includes Binary indicators, likely representing whether an email is classified as spam (1) or not spam/ham (0),

Training the Spam Detection Model:

A suitable machine learning algorithm is chosen for training the spam detection model. We have used Naive Bayes algorithm

The data set is split into training and testing sets on a specified percentage, 70% for the training set and 30% for testing.

The Naive Bayes model (naive) is trained using the training dataset.

The program then predicts the class labels for the testing dataset and stores these predictions.

APPLICATIONS

The paper on a data science approach to email spam classification using machine learning techniques provides a number of applications and implications at the organizational and individual levels:

Advanced email management: By accurately classifying emails as spam or non-spam (ham), the system helps to manage email traffic more efficiently. This reduces the clutter in an individual's or organization's inbox by simply absorbing unwanted emails.

Increased efficiency: By reducing the amount of manual input required by spammers, the system can significantly increase efficiency. Employees spend less time dealing with irrelevant emails, which can otherwise distract them from more important work.

Enhancing security: The system's ability to distinguish between spam and ham emails plays an important role in enhancing security. Spam emails often contain malicious links, scams, or phishing attempts. Effective spam filtering protects users from this potential threat by blocking harmful emails before they reach the inbox.

Cost optimization: Reducing spam can also help reduce IT costs. Spam takes up server space and can slow down network performance. By effectively managing spam, organizations can save on storage costs and maintain better overall IT efficiency.

SNIPPETS OF CODE:

```
0
1 int main()
2 {
3     std::srand (unsigned(std::time(0)));
4     std::ifstream in;
5     in.open("Tf Idf for cpp.csv");
6     NaiveBayes naive = NaiveBayes();
7     vector<vector<float>> dataset;
8     vector<float> row;
9     int index=0;
10    string temp,line;
11    float temp2;
12
13    while(std::getline(in,line))
14    {
15        std::cout<<index+1<<"\r";
16        std::cout.flush();
17        index++;
18        temp="";
19        printf("hi");
20
21        for(std::size_t i=0;i<line.size();i++)
22        {
23            if(line[i]!='\n')
24            {
25                temp = temp + line[i];
26                printf("temp for");
```

```
bool Preprocessing::checkSpecialChar(char ch)
```

```
{
```

```
    switch(ch)
```

```
    {
```

```
        case '0':
```

```
        case '1':
```

```
        case '2':
```

```
        case '3':
```

```
        case '4':
```

```
        case '5':
```

```
        case '6':
```

```
        case '7':
```

```
        case '8':
```

```
        case '9':
```

```
        case '!':
```

```
        case '\"':
```

```
        case '#':
```

```
        case '$':
```

```
        case '%':
```

```
        case '&':
```

```
        case '\\':
```

```
        case '(':
```

```
        case ')':
```

```
        case '*':
```

```
        case '+':
```

```
        case ',':
```

```
        case '-':
```

```
        case ' ':
```



```

8  typedef struct class_summary
12
13  } class_summary;
14
15  class_summary calculateClassSummary(std::vector<std::vector<float>> dataset, float class_label);
16  float prob_By_Summary(const std::vector<float> &test_data ,const class_summary &summary );
17
18  class NaiveBayes
19  {
20      private:
21          std::vector<class_summary> Summary;
22          std::vector<float> unique_label;
23
24      public:
25          void fit(std::vector<std::vector<float>> dataset);
26          int predict(const std::vector<float>& test_data);
27  };
28
29
30  void NaiveBayes::fit(std::vector<std::vector<float>> training_data )
31  {
32      unique_label = training_data[training_data.size()-1];
33      std::sort(unique_label.begin(),unique_label.end());
34      unique_label.erase(std::unique(unique_label.begin(),unique_label.end()),unique_label.end());
35      cout<<"Unique label  : "<<unique_label.size()<<endl;
36      for (auto row = unique_label.begin(); row != unique_label.end(); row++)
37      {
38          Summary.push_back(calculateClassSummary(training_data, *row));
39      }

```

```

11 void BagOfWords::transform(std::vector<std::vector<std::string>> processedData)
12 {
13
14     std::vector<std::string> uniquewords;
15     std::unordered_map<std::string,int> m;
16
17     std::ifstream in;
18     in.open("05 - unique words.txt");
19     std::ofstream out;
20     out.open("08 - BagOfWords.csv");
21
22     std::string str;
23     int z=0;
24     while(in>>str)
25     {
26         std::cout<<"Reading...."<<z+1<<" "<<str<<std::endl;
27         z++;
28         uniquewords.push_back(str);
29     }
30     //should be 21768 unique words
31     std::cout<<"Unique words : "<<uniquewords.size()<<std::endl;
32
33     //2. writing the columns in the output BagOfWords.csv file
34     for(int i=0;i<uniquewords.size();i++)
35     {
36         out<<uniquewords[i]<<",";
37     }
38     out<<std::endl;
39

```

EXPECTED OUTPUT:

1. Preprocessing
2. Bag of words
3. Naive Bayes Classifier

Challenges

The paper on the application of machine learning techniques for email spam classification presents several challenges that may affect the efficacy and applicability of the proposed system:

Scalability: As the number of users and the number of emails increases, the system must scale without losing performance or speed. Scalability challenges include handling high data loads and maintaining fast uptime.

Privacy concerns: Using machine learning in email classification must ensure the privacy of users' data. Analyzing and storing emails can raise data security and user privacy concerns, and requires strong security measures to protect sensitive information.

Integrating existing configurations: Integrating a new spam filtering solution into an existing email infrastructure can be difficult. For widespread adoption, it needs to be compatible with different email systems, operating systems and user interfaces.

User-specific flexibility: The document specifies a system that adapts to an individual user's configuration. However, it can be difficult to create a well-personalized image without user overload or privacy compromise.

These challenges highlight the importance of continuous research, development and modification of spam filtering systems to respond to the ever-changing threat landscape, while balancing performance, usability and security

Future Directions and Enhancements

Future Directions and Improvements in Email Spam Classification Systems Using Machine Learning Tools Incorporating real-time learning capabilities including exploring advanced machine learning models such as neural networks and ensemble methods increase accuracy and efficiency can help the system adapt to other spam methods more quickly. Increasing personalization to reduce false positives and negatives, and improving feature engineering to identify more predictive email attributes are also key areas for expanding spam detection and enhancing data protection measures on other digital communication systems is critical to protecting user privacy and data integrity Its role will be expanded to ensure compliance with international data protection laws. Furthermore, by combining teaching implementation and feedback techniques, the system will be robust and user-friendly by improving scalability across platforms and devices This development aims not only to improve technical capacity but also to address usability, security and compliance aspects.

REFERENCES

- [1] G, Savitha, et al. “Advanced Email Spam Detection: A Machine Learning Solution.” 2023 International Conference on Evolutionary Algorithms and Soft Computing Techniques (EASCT), Evolutionary Algorithms and Soft Computing Techniques (EASCT), 2023 International Conference On, Oct. 2023, pp. 1–5. EBSCOhost, <https://doi.org/10.1109/EASCT59475.2023.10393777>.
- [2] Dhar, Akanksha, et al. “Comparative Analysis of Deep Learning, SVM, Random Forest, and XGBoost for Email Spam Detection: A Socio- Network Analysis Approach.” 2023 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS), Computing, Communication, and Intelligent Systems (ICCCIS), 2023 International Conference On, Nov. 2023, pp. 701–07. EBSCOhost, <https://doi.org/10.1109/ICCCIS60361.2023.10425771>.
- [3] Shah, Syed Attique, et al. “Sorting the Digital Stream: Big Data-Driven Insights into Email Classification for Spam and Ham Detection.” 2023 IEEE International Conference on Big Data (BigData), Big Data (BigData), 2023 IEEE International Conference On, Dec. 2023, pp. 5598–607. EBSCOhost, <https://doi.org/10.1109/BigData59044.2023.10386224>.

We did additional research on other similar papers to find difference in the literature and methodology between our proposed topic and the articles to further emphasize our own research and efforts.

CONCLUSION

Through the analysis of email headers, content, and relevant features, the model demonstrates the capability to distinguish between spam and ham emails with a high level of accuracy.

The approach gives a solution for email spam classification, showcasing the potential of machine learning techniques in enhancing email security and user experience.

By using advanced machine learning techniques, the system minimizes the occurrence of false positives and false negatives, ensuring that legitimate emails reach users while spam is effectively filtered out.